

GAME TEST REPORT

test 2

Pre-Certification Self-Assessment

ISSUES FOUND

Report Number	BS-SLOTS-20260808-CDD1E66D
Date of Report	2026-08-08
Issued By	BlueStuff Game Studio · Automated Test Engine
Report Recipient	6a43beed9eae7b49adc60e
Engine Under Test	slots
Standard Tested Against	GLI-19 v3.0 — Interactive Gaming Systems
Classification	Pre-certification self-assessment (Non-Jurisdictional)
Software Fingerprint	sha256:2457a837740120b17d5e54097781ba9ee2afd6d6... · 62 files
Result Summary	2 passed · 1 failed · 1 warnings · 9 manual · 0 source findings

Disclaimer. This is an automated pre-certification self-assessment produced by BlueStuff Game Studio's own test engine, published unmodified — including any failures. It is tested *against* the GLI-19 v3.0 technical standard but is **not** an accredited-laboratory certification and does not imply affiliation with, or endorsement by, Gaming Laboratories International. Final certification requires an independent accredited testing laboratory.

What this report is

This is an independent, automated fairness test of **test 2**. A test engine played the game millions of times and measured whether it behaves fairly: is it truly random, does it pay back what it should, and can anyone break it? The results below are in plain language, and **every result links to the raw evidence** behind it — so nothing here has to be taken on trust.

1. Summary — what we found

This build did not pass every check. Details and reproduction steps are below.

All the evidence for this report: [open the evidence folder](#).

Evaluator's Assessment

The reasoned opinion of the automated evaluator, based strictly on the measured results in this report.

GLI-19 Compliance Evaluation — Game 55016df4 (Slots Engine)

Run ID: cdd1e66d-6ba1-4c6c-9ab3-4a7cdb18e546 | Seed: gli-1786181394271 | Date: 2026-08-08

Overall Determination: Not Compliant

This game does not meet GLI-19 certification requirements at this time. A hard FAIL on §3.2.2 (RNG uniformity) blocks certification regardless of the status of other clauses. Additionally, the §4.7.1 RTP discrepancy is unresolved, all custom-case verification is deferred (MANUAL), and no live-session testing was possible. The game cannot be certified until at minimum the §3.2.2 defect is corrected and re-audited.

§3.2.2 — RNG Statistical Tests: FAIL

Of six applicable tests (interplay-correlation was N/A for single-value draws), five passed comfortably. The **coupon-collector** test failed at the 99% family-wise confidence level (Fisher-combined $p = 4.04 \times 10^{-4}$; Holm–Bonferroni threshold 1.67×10^{-3}) across 10 million draws over five independent seeds. The evidence attributes this to the RNG's **uniform-scaling method** (`rng.int(0, 15)`): draws do not cycle through the full output alphabet with the frequency a truly uniform generator would produce. This is a structural RNG defect, not a sampling artefact — the wide spread of per-replicate p-values (0.00078 to 0.493) confirms it is intermittent but real and reproducible with the same seed. For players, this means symbol sequences are not provably fair; for operators, it means the game cannot obtain a GLI certificate in this state.

§3.2.3 / §3.2.4 — Symbol Distribution & Independence: PASS

Final symbol frequencies across all 11 symbols conform to the declared "high" weight profile (chi-square Fisher combined $p = 0.2614$ over 10 M rounds). Draw independence — serial correlation, overlapping-tuples, and inter-reel correlation — all pass at the 99% level (Fisher combined $p = 0.4744$ over 35 M draws). These results are well-powered and credible.

§4.7.1 — RTP: WARN

The measured RTP of **98.345%** (10 M rounds, SE = 0.333 pp, 3σ interval 97.346%–99.345%) exceeds the **declared target of 96.50%** by 1.845 pp — more than 3 standard errors. The GLI-19 §4.7.1 75% floor is met, so this is not a floor violation. However, the gap between measured and declared RTP is large enough that one of two explanations must be resolved before certification:

- The declared target is **mis-stated** in the game documentation; or
- The 10 M-round sample is **insufficient** to converge for this game's volatility profile (the max-win cap of 5000× was reached once in 10 M rounds, indicating significant tail weight).

A further complication: the custom-case evidence references **two different declared RTPs** — 96.50% and 94.0636% — across different custom-case IDs. This inconsistency in the declared figures must be resolved by the developer before any RTP audit result can be considered conclusive.

§4.7.3 — Jackpot / Max-Win Frequency: MANUAL ♦

The 5000× cap was reached once in 10 M rounds, which is informative but insufficient to establish the cap's true frequency to the precision §4.7.3 requires (one occurrence per 100 M games at minimum). A dedicated long-run simulation at ≥ 100 M rounds is needed. The two custom cases covering base-game and buy-bonus cap enforcement are also MANUAL because the backend is not a template backend; they were not executed and cannot be considered audited.

Custom Cases (§4.7.1 & §4.7.3): All MANUAL ♦

All eight custom-case verifications — covering base-game RTP, buy-bonus RTP, base-game max-win cap, and buy-bonus max-win cap — returned MANUAL because the deployed backend does not conform to the game-engine-template contract the harness requires. None of these cases were executed; their outcomes are **unverified**. This is a coverage gap, not a finding of failure, but it means the buy-bonus RTP and cap enforcement have not been independently verified.

Adversarial Findings: None

The adversarial probe suite returned no findings. Runtime probes (double-spend, cash-out-after-settlement, session fixation) require a containerised backend and were not run; those attack classes remain untested.

Recommendations (Priority Order)

1. **Fix the RNG (blocker).** Replace the `rng.int(0, 15)` uniform-scaling implementation with a method that passes the coupon-collector test. After the fix, re-run `gli-audit §3.2.2` at full scale (10 M draws, 5 replicates) before any other certification work proceeds.
2. **Reconcile the declared RTP (blocker).** Determine the single authoritative declared RTP — 96.50% or 94.0636% — correct the game documentation, and re-run the §4.7.1 measurement at a larger sample (≥ 50 M rounds recommended given the observed volatility) to establish whether the measured RTP converges to the declared figure.
3. **Run a dedicated long-run §4.7.3 simulation.** A ≥ 100 M-round run is needed to establish the 5000× cap frequency. Engage the harness team to schedule a large-scale simulation pass.
4. **Expose the real backend to the custom-case harness.** The game-engine-template contract gap means eight critical custom cases — including buy-bonus RTP and cap enforcement — have never been exercised. Work with the backend team to either (a) surface a template-compatible test harness endpoint, or (b) provide test seeds that allow the harness to drive the live backend deterministically.
5. **Enable live-session testing.** Provide operator / brand / game-code parameters so the live demo path can be tested end-to-end. This is lower priority than items 1–4 but required for a complete audit record.

2. Results at a glance

Each row is one thing we checked, in plain terms, with a link to the evidence behind it.

What we checked	Result	Based on	Evidence
Randomness quality	 Did not pass	10 million rounds	View
Even distribution	 Passed	10 million rounds	View
Independence of rounds	 Passed	35 million rounds	View
Payout fairness (RTP)	 Needs attention	10 million rounds	View
Top award reachable	 Needs human review	10 million rounds	View
Operator check (4.7.3)	 Needs human review	—	View
Operator check (4.7.3)	 Needs human review	—	View
Operator check (4.7.1)	 Needs human review	—	View
Operator check (4.7.1)	 Needs human review	—	View
Operator check (4.7.3)	 Needs human review	—	View
Operator check (4.7.3)	 Needs human review	—	View
Operator check (4.7.1)	 Needs human review	—	View
Operator check (4.7.1)	 Needs human review	—	View
Attempts to break the game	 Nothing found	source + probes	View

3. What we tested, in plain terms

Randomness quality Did not pass

What we checked. We looked at whether the random-number generator behaves like true randomness, with no pattern a player or the operator could predict or exploit.

What we found. Usage "uniform scaling" (rng.int(0, 15)), 5 independent seeds combined by Fisher's method. 6 of the 7 tests named in §3.2.2 applied, 1 not applicable. 1 rejected at the 99% family-wise confidence level: coupon-collector ($p = 4.037e-4$, threshold $1.667e-3$).

Why it matters. If outcomes were predictable, the game would not be fair to either side. (GLI-19 3.2.2) · [see the full evidence](#)

Even distribution Passed

What we checked. We looked at whether every possible outcome comes up as often as it should — none too often, none too rarely.

What we found. Final outcome output over 11 symbols conforms to the declared weight profile "high" across 5 independent seeds (Fisher combined chi-square = 12.36, df = 10, $p = 0.2614$; worst single seed $p = 0.0087$). Symbols: chalice, crown, gem_blue, gem_green, gem_purple, gem_red, gem_yellow, hourglass, multiplier, ring, scatter.

Why it matters. A fair game needs every symbol or result to appear at its intended rate. (GLI-19 3.2.3) · [see the full evidence](#)

Independence of rounds Passed

What we checked. We looked at whether each round stands on its own, with earlier results never nudging later ones.

What we found. Independence between successive draws (serial correlation, lag-1 contingency) and within a single draw across 5 reels (interplay correlation), over 5 independent seeds. 3 of the 7 tests named in §3.2.2 applied. None rejected at the 99% family-wise confidence level (Holm–Bonferroni; Fisher combined $p = 4.744e-1$).

Why it matters. Past spins must not influence future spins — there are no hidden streaks the game steers. (GLI-19 3.2.4) · [see the full evidence](#)

Payout fairness (RTP) Needs attention

What we checked. We looked at whether, over millions of rounds, the game returns the share of stakes it is designed to (its Return To Player).

What we found. Measured RTP 98.345% over 10,000,000 rounds (3-sigma interval 97.346%–99.345%, SE 0.3331pp). Declared target 96.50%. GLI-19 §4.7.1 floor is 75%: met. Measured RTP differs from the declared target by 1.845pp, more than 3 standard errors — either the sample is too small for this volatility, or the target is mis-declared. Hit rate 25.51%, feature rate 0.374%, max observed win 5000.0x against a 5000x cap (cap reached 1 times).

Why it matters. This is the headline promise to players — that the game pays back what it advertises. (GLI-19 4.7.1) · [see the full evidence](#)

Top award reachable Needs human review

What we checked. We looked at whether the biggest advertised win can actually be hit.

What we found. §4.7.3 requires the highest advertised award to occur at least once in 100,000,000 games unless prominently disclosed. The max-win cap is 5000x and was reached 1 times in 10,000,000 rounds. Establishing the cap frequency to the precision this clause needs requires a run several orders of magnitude larger; escalate to a dedicated long-run simulation.

Why it matters. An advertised top prize that can never land would mislead players. (GLI-19 4.7.3) · [see the full evidence](#)

Operator check (4.7.3) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Base game win never exceeds 5000x cap — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.3) · [see the full evidence](#)

Operator check (4.7.3) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Buy-Bonus win never exceeds 5000x cap — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.3) · [see the full evidence](#)

Operator check (4.7.1) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Buy-Bonus RTP matches declared 96.5% — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.1) · [see the full evidence](#)

Operator check (4.7.1) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Base RTP matches declared 96.5% — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.1) · [see the full evidence](#)

Operator check (4.7.3) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Base-game max win does not exceed declared 5000x cap — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.3) · [see the full evidence](#)

Operator check (4.7.3) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Buy-bonus max win does not exceed declared 5000x stake cap — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.3) · [see the full evidence](#)

Operator check (4.7.1) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Buy-bonus RTP matches declared 94.0636% — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.1) · [see the full evidence](#)

Operator check (4.7.1) Needs human review

What we checked. We looked at an additional check the operator defined for this game.

What we found. [Custom] Base-game RTP matches declared 94.0636% — custom cases run against the game-engine-template contract, but this backend is not a template backend, so it was not executed here.

Why it matters. Operators can add their own pass/fail conditions on top of the standard tests. (GLI-19 custom:4.7.1) · [see the full evidence](#)

4. Attempts to break the game

Beyond the maths, we probed the game for the ways games actually get exploited — unfair logic, advertised prizes that can't be won, and fairness claims with nothing behind them. **None of these probes found an issue in this run.**

Some live-system attacks (for example placing the same bet twice at once, or cashing out after a round is settled) can only be tested against a running server; where those were not run against a live instance they are out of scope for this report rather than assumed safe.

5. The live deployed game

We also try to test the **actual game running live on the server** by playing real rounds through it. For this run that could not be done:

Launch parameters (operator / brand / game code) are not available for this game, so a live demo session could not be minted.

6. The evidence — and how to check it yourself

Every result above links to a readable evidence file in the folder for this report. Each file carries the exact numbers, the random seed, and enough detail to check the result yourself.

Evidence file	What it contains
3.2.2.md	What was tested for requirement 3.2.2, how it was tested, and the full measured result.
3.2.3.md	What was tested for requirement 3.2.3, how it was tested, and the full measured result.
3.2.4.md	What was tested for requirement 3.2.4, how it was tested, and the full measured result.
4.7.1.md	What was tested for requirement 4.7.1, how it was tested, and the full measured result.
4.7.3.md	What was tested for requirement 4.7.3, how it was tested, and the full measured result.
custom-4.7.3.md	What was tested for requirement custom:4.7.3, how it was tested, and the full measured result.
custom-4.7.3-cdd1.md	What was tested for requirement custom:4.7.3, how it was tested, and the full measured result.
custom-4.7.1.md	What was tested for requirement custom:4.7.1, how it was tested, and the full measured result.
custom-4.7.1-cdd1.md	What was tested for requirement custom:4.7.1, how it was tested, and the full measured result.
custom-4.7.3-cdd1.md	What was tested for requirement custom:4.7.3, how it was tested, and the full measured result.
custom-4.7.3-cdd1.md	What was tested for requirement custom:4.7.3, how it was tested, and the full measured result.
custom-4.7.1-cdd1.md	What was tested for requirement custom:4.7.1, how it was tested, and the full measured result.
custom-4.7.1-cdd1.md	What was tested for requirement custom:4.7.1, how it was tested, and the full measured result.
adversarial-findings.md	Every "attempt to break the game" probe and what it found (empty if none).
live-tests.md	Runtime checks played against the live deployed game — double-spend, session/validation, and the deployed-vs-certified RTP cross-check.
checksums.md	SHA-256 of each of the 62 tested source files, plus the aggregate fingerprint this report is bound to.
README.md	Plain-language index of this evidence folder and how to verify results.

Reproduce it yourself. Every result is reproducible from the random seed `gli-1786181394271`. The test engine has no external dependencies and no AI in its numeric path — every statistic is computed by reviewable, in-tree code, so re-running against the same engine with this seed reproduces the same numbers exactly.

7. Software fingerprint

We took a SHA-256 checksum of every one of the 62 source files tested. The single fingerprint below is a hash over all of them — change any tested file and it changes too, so this report is locked to exactly the code that was tested.

Aggregate fingerprint: sha256:2457a837740120b17d5e54097781ba9ee2afd6d60ac3eb27b55bc9ff445985d0

Full per-file checksums: [checksums.json](#) .

8. Test Environment

Test engine	SportsITDev/gli-audit-engine
Runtime	Node v24.18.0
Seed	gli-1786181394271
Started	2026-08-08T09:29:54.366Z
Finished	2026-08-08T09:37:00.645Z
Generated by	gli-audit-engine game-tester agent